

Herencia múltiple y sus usos

La herencia múltiple es una característica de Python que permite que una clase herede de más de una clase base.

Esto es útil porque nos ayuda a reutilizar código y a crear modelos más completos, combinando diferentes comportamientos en una sola clase.

Por ejemplo:

Imagina que tienes una clase `CoffeeMaker`. Podría heredar de una clase `HotWaterProvider` para atender a clientes que solo quieren té, **y al mismo tiempo** mantener su función principal de hacer café.

Usos de la herencia múltiple

- **Modelado de objetos complejos**

Permite representar objetos del mundo real que cumplen más de un rol.

Ejemplo: un `TrabajadorEstudiante` puede heredar de la clase `Trabajador` y de la clase `Estudiante`.

- **Reutilización de código**

Si tienes varias clases con funcionalidades comunes, puedes combinarlas.

Ejemplo: una clase `Logger` que escribe en **HTML** y otra que escribe en **JSON**. Una clase `CombinedLogger` puede heredar de ambas para tener la capacidad de guardar registros en los dos formatos.

- **Mixins (comportamientos adicionales)**

Un **Mixin** es una clase pequeña que agrega una funcionalidad extra a otra clase.

Ejemplo: un `ReproductorDeSonido` puede ser un mixin que se añade tanto a la clase `Pez` como a la clase `Pájaro`, dándole a cada uno la capacidad de reproducir sonidos distintos.

- **Sistemas de autenticación**

Una clase puede heredar de varias formas de autenticación, como por **usuario y contraseña** o por **token**, combinando diferentes mecanismos de seguridad.

- **Clases con múltiples comportamientos**

Imagina una clase `Vehiculo` que pueda comportarse como `Coche` (conducir) y también como `Barco` (navegar). Gracias a la herencia múltiple, `Vehiculo` puede heredar de ambas y combinar sus funciones en un mismo objeto.